Homework I

- La soluzione degli esercizi *(ovvero un PDF contenente la risoluzione analitica degli esercizi ed il codice delle funzioni sviluppate nell'Esercizio 3)* deve essere caricata sul portale del corso entro le ore 23:59 di domenica 17 Novembre 2024, sotto il nome di Homework1. Il voto massimo in caso di consegna in ritardo è diminuito di 1 punto. In ogni caso, il PDF finale deve essere consegnato entro 5 giorni dalla data dell'esame orale.
- Le qualità dell'esposizione, la capacità di sintesi e la chiarezza del documento finale rientrano nella valutazione dell'homework. La scrittura del documento finale in Latex o in qualsiasi altro formato elettronico è fortemente incoraggiata. Se il documento finale è scritto a mano deve essere facilmente leggibile.
- La collaborazione in gruppi (fino a un massimo di 5 persone) e lo scambio di idee sono incoraggiati. In ogni caso, ogni studente deve sottomettere una copia del documento finale (in formato PDF) e del codice numerico, e specificare con chi ha collaborato e per quale specifica parte del lavoro.

Esercizio 1. Consider the network in Figure 1 with link capacities

$$c_1 = c_3 = c_5 = 3, \qquad c_6 = c_7 = 1 \qquad c_2 = c_4 = 2.$$

(a) Compute the capacity of all the cuts and find the minimum capacity to be removed for no feasible flow from o to d to exist.

(b) You are given $x > 0$ extra units of capacity ($x \in \mathbf{Z}$). How should you distribute them in order to maximize the throughput that can be sent from o to d? Plot the maximum throughput from o to d as a function of $x \geq 0$.

(c) You are given the possibility of adding to the network a directed link e_8 with capacity $c_8 = 1$ and $x > 0$ extra units of capacity ($x \in \mathbf{Z}$). Where should you add the link and how should you distribute the additional capacity in order to maximize the throughput that can be sent from o to d? Plot the maximum throughput from o to d as a function of $x \geq 0$.

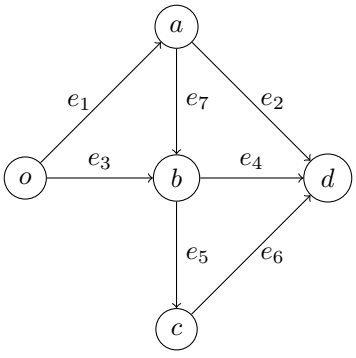


Figure 1

Esercizio 2. Consider o-d network flows on the graph in Figure 2. The links are endowed with delay functions

$$\tau_1(x) = \tau_6(x) = 3x\,, \quad \tau_2(x) = \tau_4(x) = x + 1\,, \quad \tau_3(x) = 3, \quad \tau_5(x) = 2x + 2, \tag{1}$$

and the throughput is 2.

(a) Compute the social optimum flow vector, i.e., the flow vector that minimizes the average delay from o to d.

(b) Compute the user optimum flow vector, i.e., the Wardrop equilibrium, and the price of anarchy.

(c) Consider a new link e_7 with delay function $\tau_7(x) = x$. Find a head and a tail of the link e_7 such that Braess' paradox arises, and compute the price of anarchy on the new graph.

(d) Compute an optimal toll vector ω on the new graph, i.e., a non-negative toll vector that reduces the price of anarchy to 1. If possible, compute a full-support optimal toll vector, i.e., such that $\omega_e > 0$ for every link e. Construct an optimal toll vector with the smallest possible support.

(e) Consider the original network in Figure 2. Assume that

$$\tau_3(x) = \alpha x + 1,$$

with $\alpha \geq 0$, and $\tau_e(x)$ are as defined in (1) for all $e = 1, 2, 4, 5, 6$. Consider o-d network flows on the new graph with throughput $\chi > 0$. Find an optimal toll vector independent of χ and α. *Hint*: focus on the optimization problems related to social optimum flows and Wardrop equilibria flows.

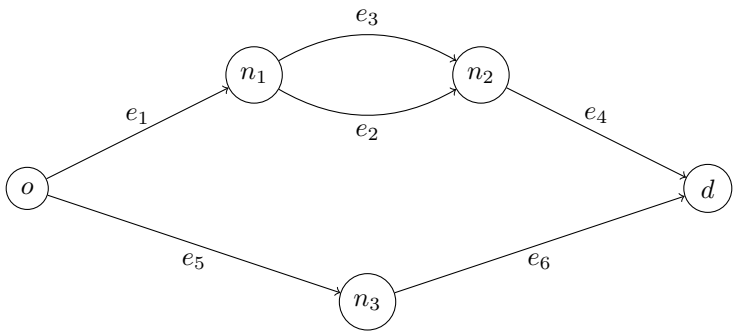


Figure 2

Esercizio 3. Consider the simple graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ in Figure 3.

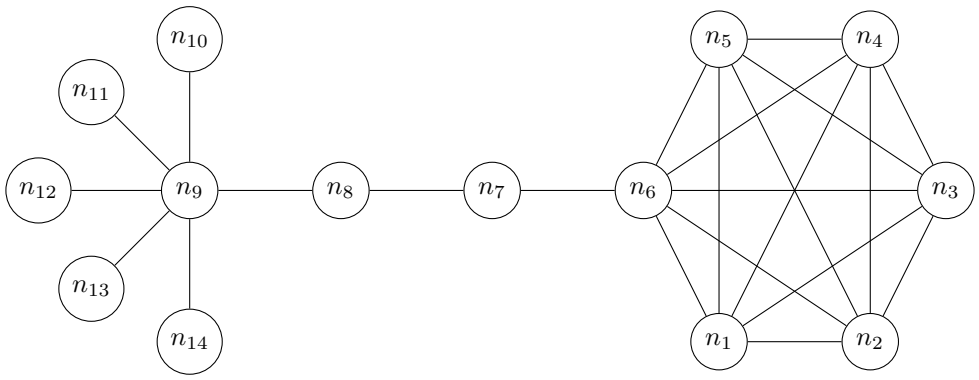


Figure 3

(a) Compute the degree centrality, the eigenvector centrality, the invariant distribution centrality, and comment the results. You can implement the computation in Matlab or Python.

(b) Compute the Katz centrality, with $\beta = 0.15$ and uniform intrinsic centrality μ. You can implement the computation in Matlab or Python.

(c) Write a *distributed* algorithm in Matlab or Python for the computation of Page-rank centrality, with $\beta = 0.15$ and uniform intrinsic centrality μ.

(d) Explain the results of points (b) and (c), focusing on the centralities of nodes n_6 and n_9.



Politecnico di Torino

Dipartimento di Scienze Applicate e Tecnologia
MSc in Physics of Complex Systems

Dinamiche su network: Homework 1

Autori:

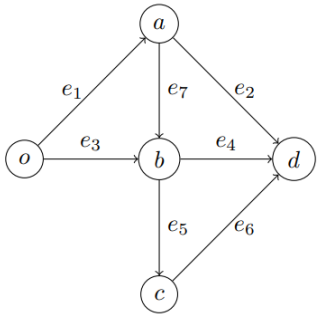
Malcangi Paolo - s331577 Alessandro Iannitti - s328784 Gaetano Ferrero - s329155

December 20, 2025

1 Esercizio 1

1.1 punto a

La capacità del minimo taglio rappresenta il minimo costo necessario per disconnettere origine e destinazione. Calcoliamo tutti i tagli possibili in questo grafo:



- $U = \{o, a, b, c\}, U^C = \{d\} \rightarrow C_U = 5$
- $U = \{o, b, c\}, U^C = \{a, d\} \rightarrow C_U = 6$
- $U = \{o, a\}, U^C = \{b, c, d\} \rightarrow C_U = 6$
- $U = \{o\}, U^C = \{a, b, c, d\} \rightarrow C_U = 6$
- $U = \{o, a, b\}, U^C = \{c, d\} \rightarrow C_U = 7$
- $U = \{o, c\}, U^C = \{a, b, d\} \rightarrow C_U = 7$
- $U = \{o, b\}, U^C = \{a, c, d\} \rightarrow C_U = 8$
- $U = \{o, a, c\}, U^C = \{b, d\} \rightarrow C_U = 7$

La minima capacità che possiamo rimuvere per fare in modo che non ci siano flussi ammissibili è quella relativa al primo taglio, con capacità di taglio uguale a

$$C_U^* = 5 \tag{1}$$

1.2 punto b

Per decidere dove inserire le unità di capacità extra, analizziamo i tagli con throughput inferiore a 7. Li suddividiamo in due gruppi, in base agli archi comuni coinvolti:

- **famiglia A** (arco comune e_4)
  - $U = \{o, a, b, c\}, U^C = \{d\} \rightarrow C_U = 5$
  - $U = \{o, b, c\}, U^C = \{a, d\} \rightarrow C_U = 6$
- **famiglia B** (arco comune e_3)
  - $U = \{o, a\}, U^C = \{b, c, d\} \rightarrow C_U = 6$
  - $U = \{o\}, U^C = \{a, b, c, d\} \rightarrow C_U = 6$

La famiglia A è composta dall'unione dei tagli accomunati dalla presenza dell'arco e_4. La famiglia B presenta invece e_3 come arco comune. Dato che, se aggiungessimo capacità al taglio di capacità minima, il min-cut tenderebbe a variare, la nostra strategia deve essere quella di incrementare il min-cut e, contemporaneamente, cercare di aumentare la capacità di tutti i tagli. Incrementiamo progressivamente le capacità sugli archi e_4 ed e_3, essendo gli archi associati alle due famiglie precedentemente definite.
Questi due archi sono anche presenti in ogni taglio con $C_U > 6$, quindi non rappresentano un problema e possono

essere ignorati. Procedendo dunque in questa direzione, otteniamo una step function ($\tau : \mathbb{N} \to \mathbb{N}$) definita in questo modo:

$$\tau(x) = 5 + \sum_{x=1}^{\infty} g(x) \quad g(x) = \begin{cases} 0 & \text{if x = even} \\ 1 & \text{if x = odd} \end{cases} \tag{2}$$

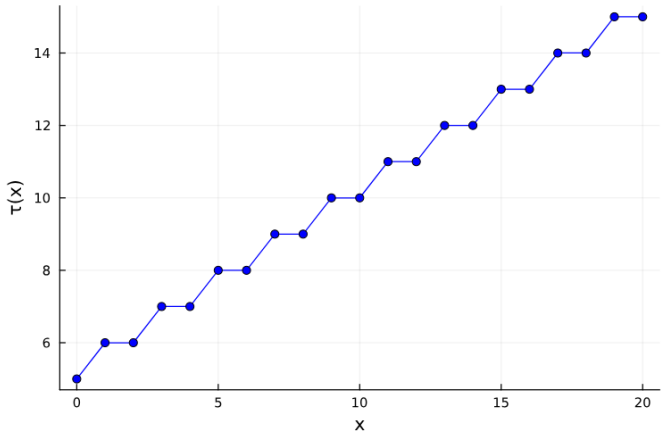


Figure 1: Throughput in funzione di x

1.3 punto c

Se ci fosse concessa la possibilità di aggiungere un arco con capacità unitaria tra due nodi qualsiasi, la scelta più logica sarebbe di certo quella di inserirlo direttamente tra l'origine e la destinazione. In questo modo, l'arco e_8 sarebbe in comune tra tutti i tagli e, per aumentare il massimo throughput, basterebbe aggiungere tutte le unità di capacità extra nel suddetto arco. In questo caso, la funzione sarebbe lineare

$$\tau(x) = 5 + x \tag{3}$$

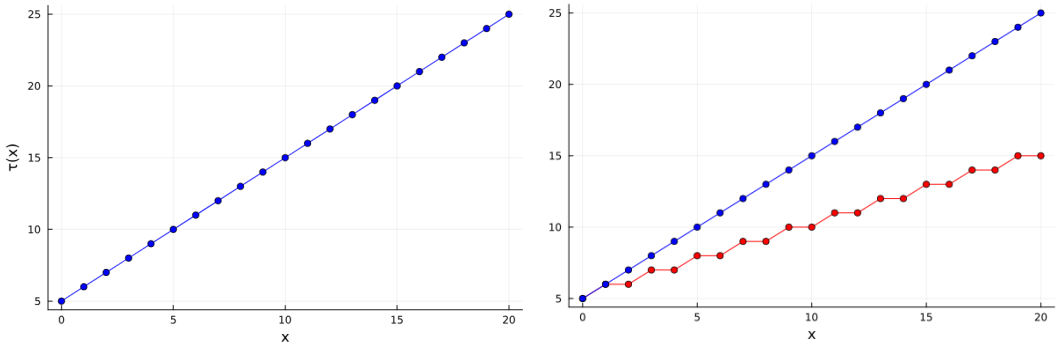


Figure 2

Nel grafico a destra in figura 2 è mostrata una comparazione rispetto il caso precedente, in cui non era presente l'arco e_8.

2 Esercizio 2

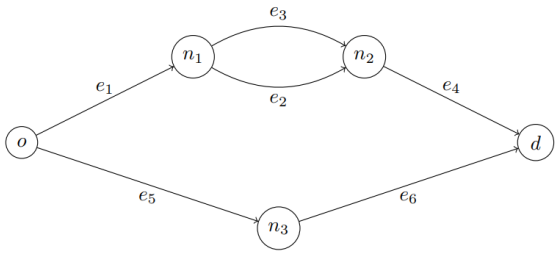


Figure 3

2.1 punto a

Dobbiamo risolvere il problema dell'ottimizzazione dei flussi

$$M(\nu) = \min_{f \geq 0; Bf = \nu} \sum_{e \in \mathcal{E}} \psi_e(f_e) \tag{4}$$

Dove il throughput è $\tau = 2$ e la funzione costo è definita come

$$\psi_e(f_e) = \tau_e(x) f_e(x) \tag{5}$$

Tenendo conto dei ritardi assegnati ad ogni arco e delle equazioni di bilancio dei flussi:

$$\begin{cases} f_1 + f_5 = 2 \\ f_1 = f_2 + f_3 \\ f_4 = f_1 \\ f_5 = f_6 \end{cases} \tag{6}$$

arriviamo a dover minimizzare una funzione a due variabili $G(f_1, f_2)$ così definita:

$$G(f_1, f_2) = 9f_1^2 + f_2^2 - 18f_1 - 2f_2 + 24 \tag{7}$$

Trattandosi di una parabola, non sarà necessario valutare l'Hessiana. Ci basterà calcolare le derivate parziali rispetto i due flussi per concludere che il punto stazionario è in corrispondenza di

$$(f_1^*, f_2^*) = (1, 1) \tag{8}$$

Considerando i vincoli in (6), il vettore dei flussi ottimale sarà

$$f = (1, 1, 0, 1, 1, 1) \tag{9}$$

2.2 punto b

Il Wardrop equilibrium è possibile calcolarlo in due modi differenti.
Nel primo, sftuttiamo la sua definizione: il tempo di percorrenza su ciascun percorso utilizzato da una frazione non nulla di utenti deve essere il minimo

$$z_p > 0 \implies \sum_{e \in \mathcal{E}} A_{ep}^{(o,d)} \tau_e(f_e^{(0)}) \leq \sum_{e \in \mathcal{E}} A_{eq}^{(o,d)} \tau_e(f_e^{(0)}) \quad \forall q \in \Gamma_{o,d} \tag{10}$$

Ora, osserviamo che i percorsi possibili nel grafo sono 3 e valutiamo i ritardi associati ad ognuno di essi:

$$\begin{cases} p^{(1)} & \text{cammino } e_1 - e_2 - e_4 \implies \Delta_1 = 5z_1 + 4z_2 + 2 \\ p^{(2)} & \text{cammino } e_1 - e_3 - e_4 \implies \Delta_2 = 4z_1 + 4z_2 + 4 \\ p^{(3)} & \text{cammino } e_5 - e_6 \implies \Delta_3 = 5z_3 + 2 \end{cases} \tag{11}$$

possiamo procedere per tentativi e valutare le condizioni imposte dall'equilibrio di Wardrop

- $z1 > 0$, c'è qualcuno sul percorso 1.
  In questo caso, le diseguaglianze da verificare sono

$$\begin{cases} 5z_1 + 4z_2 + 2 \leq 4z_1 + 4z_2 + 4 \implies z_1 \leq 2 \\ 5z_1 + 4z_2 + 2 \leq 5z_3 + 2 \end{cases} \tag{12}$$

- $z_2 > 0$ c'è qualcuno sul percorso 2.

$$\begin{cases} 4z_1 + 4z_2 + 4 \leq 5z_1 + 4z_2 + 2 \implies z_1 \geq 2 \\ 4z_1 + 4z_2 + 4 \leq 5z_3 + 2 \implies z_1 \leq -\frac{1}{2} \end{cases} \tag{13}$$

  chiaramente, questo flusso non risulta ammissibile, possiamo concludere che $z_2 = 0$

- $z_3 > 0$ c'è qualcuno sul percorso 3

$$\begin{cases} 5z_3 + 2 \leq 5z_1 + 4z_2 + 2 \implies z_3 \leq z_1 \\ 5z_3 + 2 \leq 4z_1 + 4z_2 + 4 \implies z_3 \leq 1 \end{cases} \tag{14}$$

Concludiamo dunque che passerà una unità di throughput nel percorso 1 e 3, mentre il percorso 2 non sarà occupato.
A questo punto, poichè il vettore dei flussi di Wardrop conicide con quello dell'ottimo sociale, possiamo considerare il price of anarchy unitario.

$$P_0A(0) = \frac{\sum_{e\in\mathcal{E}} f_e^{(0)}\tau_e(f_e^{(0)})}{\min_{f\geq 0; Bf=\nu} \sum_{e\in\mathcal{E}} f_e\tau_e(f_e)} = 1 \tag{15}$$

Avremmo potuto risolvere questo punto anche sfruttando il teorema secondo cui un vettore f è di Wardrop se e solo se è la soluzione dell'user optimal traffic assignment:

$$M(\nu) = \min_{f\geq 0; Bf=\nu} \sum_{e\in\mathcal{E}} \int_0^{f_e} \tau_e(x)dx \tag{16}$$

Procedendo con questa idea, otteniamo un risultato analogo al precedente.

2.3 punto c

Dobbiamo aggiungere un arco (e_7) al nostro grafo e verificarne gli effetti sul prezzo dell'anarchia. Il nuovo arco avrà un tempo di ritardo lineare $\tau_7(x) = x$, quindi minore dei tempi di percorrenza associati agli altri collegamenti. Per generare il paradosso di Braess, proviamo inizialmente ad aggiungere il link tra i nodi n_1 ed n_2, **questa scelta si rivelerà successivamente errata** e saremo costretti a cercare un nuovo arco.

Sfruttando il teorema descritto nel punto precedente, risolviamo il problema dell'UO-tap per trovare il vettore dei flussi di Wardrop

$$\sum_{e\in\mathcal{E}} \int_0^{f_e} \tau_e(x)dx = \frac{3}{2}f_1^2 + \frac{1}{2}f_2^2 + f_2 + 3f_3 + \frac{1}{2}f_4^2 + f_4 + f_5^2 + 2f_5 + \frac{3}{2}f_6^2 + \frac{1}{2}f_7^2 \tag{17}$$

I vincoli del nuovo problema saranno:

$$\begin{cases} f_1 + f_5 = 2 \\ f_1 = f_2 + f_3 + f_7 \\ f_4 = f_1 \\ f_5 = f_6 \end{cases} \tag{18}$$

Se li sostituiamo nella (17), otteniamo una funzione G in 3 variabili (f_1, f_2, f_7) da minimizzare tenendo conto del fatto che il flusso deve essere ammissibile.

$$G(f_1, f_2, f_7) = \frac{9}{2}f_1^2 + \frac{1}{2}f_2^2 + \frac{1}{2}f_7^2 - 8f_1 - 2f_2 - 3f_7 + 14 \quad \begin{cases} f_i \geq 0 \quad \forall i \\ f_1 \leq 2 \\ f_1 \geq f_2 + f_7 \end{cases} \tag{19}$$

Il punto critico di questa parabola è in una posizione tale da non essere ammesso all'interno del dominio, dobbiamo quindi cercare una soluzione sulla frontiera. Imponiamo il vincolo

$$f_1 = f_2 + f_7 \tag{20}$$

e lo sostituiamo nell'equazione.
In questo modo, la funzione da minimizzare diventa:

$$G(f_1, f_2) = 5f_1^2 + f_2^2 - 11f_1 + f_2 - f_1 f_2 + 14 \tag{21}$$

Otteniamo quindi il seguente vettore dei flussi di Wardrop

$$f = \left(\frac{21}{19}, \frac{1}{19}, 0, \frac{21}{19}, \frac{17}{19}, \frac{17}{19}, \frac{20}{19}\right) \tag{22}$$

Cerchiamo ora di risolvere il problema dell'ottimizzazione dei flussi tenendo conto dei nuovi vincoli riportati in (18). Questa volta la funzione da minimizzare sarà descritta da tre variabili

$$G(f_1, f_2, f_7) = 9f_1^2 + f_2^2 + f_7^2 - 18f_1 - 2f_2 - 3f_7 + 24 \tag{23}$$

L'ottimizzazione libera non ci porta ad un risultato feasible, procediamo con il metodo dei moltiplicatori di Lagrange, definendo la Lagrangiana in questo modo

$$\mathcal{L}(f_1, f_2, f_7, \lambda) = 9f_1^2 + f_2^2 + f_7^2 - 18f_1 - 2f_2 - 3f_7 + 24 + \lambda(f_1 - f_2 - f_7) \tag{24}$$

Deriviamo rispetto le variabili e risolviamo il sistema di euquazioni associato al problema.
Alla fine, ci ritroveremo con il seguente vettore dell'ottimo sociale

$$f = \left(\frac{41}{38}, \frac{11}{38}, 0, \frac{41}{38}, \frac{35}{38}, \frac{35}{38}, \frac{15}{19}\right) \tag{25}$$

Ci aspettiamo un prezzo dell'anarchia maggiore di 1

$$P_oA(0) = \frac{\sum_{e\in\mathcal{E}} f_e^{(0)}\tau_e(f_e^{(0)})}{\min_{f\geq 0; Bf=\nu} \sum_{e\in\mathcal{E}} f_e\tau_e(f_e)} \simeq 1.01026 \tag{26}$$

Sebbene il prezzo dell'anarchia sia aumentato, non abbiamo generato il paradosso di Braess perchè i tempi medi di percorrenza del nuovo grafo risulteranno minori rispetto quelli del grafo originale. Quindi

$$P_oA(0) > 1 \implies \text{paradosso di Braess} \tag{27}$$

Dobbiamo modificare la nostra scelta iniziale, provando ad interporre il link tra i nodi n_1 ed n_3, come mostrato in figura 4. A questo punto procediamo col calcolo degli ottimi

2.3.1 Vettore dei flussi di Wardrop

Ancora una volta, dovremo risolvere il problema dell'UO-tap riportato nell'equazione (17). I vincoli però saranno diversi rispetto (18)

$$\begin{cases} f_1 + f_5 = 2 \\ f_1 = f_2 + f_3 + f_7 \\ f_4 = f_2 + f_3 \\ f_6 = f_5 + f_7 \end{cases} \tag{28}$$

Inserendoli nella nostra equazione, arriviamo alla seguente funzione da minimizzare

$$G(f_1, f_2, f_3) = \frac{9}{2}f_1^2 + \frac{1}{2}f_2^2 + \frac{5}{2}f_7^2 - 8f_1 - 2f_2 + 2f_7 - 4f_1 f_7 + 14 \tag{29}$$

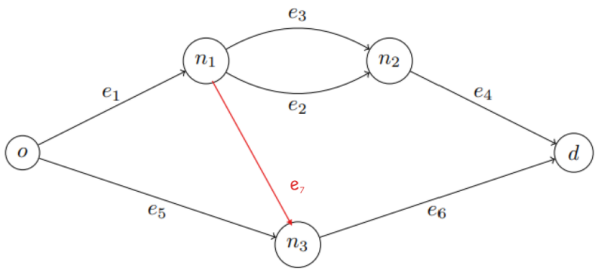


Figure 4: Caption

Dato che il minimo locale della funzione precedente non rispetta i vincoli, ovvero, si trova nel semispazio $f_7 > f_2 - f_1$, la soluzione va cercata nella frontiera del dominio ammissibile. In particolare, data la convessità della funzione e del dominio definito dai vincoli di positività dei flussi, l'ottimo della funzione va cercato nel piano $f_1 = f_2 + f_7$. Sostituendo questa condizione nella funzione definita in precedenza, otteniamo

$$G(f_1, f_2) = 3f_1^2 + 3f_2^2 - 6f_1 - 4f_2 - f_1 f_2 + 14 \tag{30}$$

Da cui è possibile ricavare il seguente vettore dei flussi di Wardrop

$$f = \left(\frac{8}{7}, \frac{6}{7}, 0, \frac{6}{7}, \frac{8}{7}, \frac{2}{7}\right) \tag{31}$$

In questo caso, a differenza del precedente, i tempi di percorrenza degli utenti sono effettivamente diminuiti, nonostante l'aggiunta di una strada. Abbiamo quindi generato il paradosso.

2.3.2 Vettore dell'ottimo sociale

Attraverso l'usuale procedura, per l'ottimo sociale troviamo un risultato analogo a quello del grafo senza l'arco e_7.

$$f = \left(1, 1, 0, 1, 1, 1, 0\right) \tag{32}$$

2.3.3 Prezzo dell'anarchia

Concludiamo quantificando il prezzo dell'anarchia

$$P_o A(0) = \frac{\sum_{e\in\mathcal{E}} f_e^{(0)} \tau_e(f_e^{(0)})}{\min_{f\geq 0; Bf=\nu} \sum_{e\in\mathcal{E}} f_e \tau_e(f_e)} \simeq 1.0086 \tag{33}$$

2.4 punto d

Sfruttando il corollario 4.4, sappiamo che è sempre possibile trovare un toll vector tale che il prezzo dell'anarchia torni ad essere unitario.

$$P_o A(\omega) = \frac{\sum_{e\in\mathcal{E}} f_e^{(\omega)} \tau_e(f_e^{(\omega)})}{\min_{f\geq 0; Bf=\nu} \sum_{e\in\mathcal{E}} f_e \tau_e(f_e)} = 1 \tag{34}$$

È possibile stimare questo vettore attraverso:

$$\omega_e = f_e^* \tau'(f_e) \tag{35}$$

Il toll vector risultante sarà quindi

$$\omega_e = (3, 1, 0, 1, 2, 3, 0) \tag{36}$$

Il toll vector appena trovato farà coincidere i vettori dell'ottimo. Dati questi pedaggi, quindi, nessuno sceglierà di proseguire verso gli archi e_3 ed e_7. Concludiamo che è possibile aumentare a piacere i pedaggi su questi archi, costruendo un toll vector con supporto pieno, senza alterare il prezzo dell'anarchia, che rimarrà unitario.

Infine, possiamo anche identificare un vettore col minimo supporto. Iniziamo a verificare che esista un supporto con una sola componente maggiore di 0.
Dai vettori trovati nel punto precedente, notiamo come il nostro intento sia fare in modo che gli utenti decidano di non proseguire verso e_7. Un'idea potrebbe essere quella di imporre un pedaggio sul solo arco e_7.
Ritorniamo al problema dell'UO-tap e cerchiamo di risolverlo considerando il pedaggio ω_7

$$\sum_{e\in\mathcal{E}}\int_0^{f_e}\tau_e(x)dx = \frac{3}{2}f_1^2+\frac{1}{2}f_2^2+f_2+3f_3+\frac{1}{2}f_4^2+f_4+f_5^2+2f_5+\frac{3}{2}f_6^2+\frac{1}{2}f_7^2+\omega_7 f_7 \tag{37}$$

Sfruttando i risultati e la metodologia descritta nel punto c, arriviamo a trovare dei minimi dipendenti dal parametro ω_7

$$\begin{cases} f_1 = \frac{f_2+6-\omega_7}{6} \\ f_2 = \frac{f_1+4+\omega_7}{6} \end{cases} \tag{38}$$

Imponendo l'uguaglianza con i punti di minimo dell'ottimo sociale ($f_1 = 1, f_2 = 1$), troviamo il pedaggio da imporre sul solo arco e_7

$$\omega_7 = 1; \quad \omega = \begin{pmatrix} 0,0,0,0,0,0,1 \end{pmatrix} \tag{39}$$

Abbiamo costruito il vettore con supporto minimo.

2.5 Punto e

Ritorniamo al grafo originale e associamo all'arco e_3 il tempo i percorrenza dato da

$$\tau_3(x) = \alpha x + 1 \tag{40}$$

Questa volta il throughput non è più pari a 2 ma ad un numero $\chi \in \mathbb{R}^+$. Ci viene chiesto se è possibile trovare un optimal toll vector indipendente da χ ed α.
A tal proposito, sfruttiamo la proposizione 4.5. L'ipotesi sulle delay functions è soddisfatta per costruzione, dobbiamo solo verificare che $A'b = \beta\mathbb{I}$

$$\iff (0,1,1,1,2,0)\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix} = 2\mathbb{I} \tag{41}$$

Abbiamo che la proposizione è verificata per $\beta = 2$, l'optimal toll vector è quindi un vettore nullo.

3 Esercizio 3

Consideriamo un grafo $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathcal{W})$ con questa struttura:

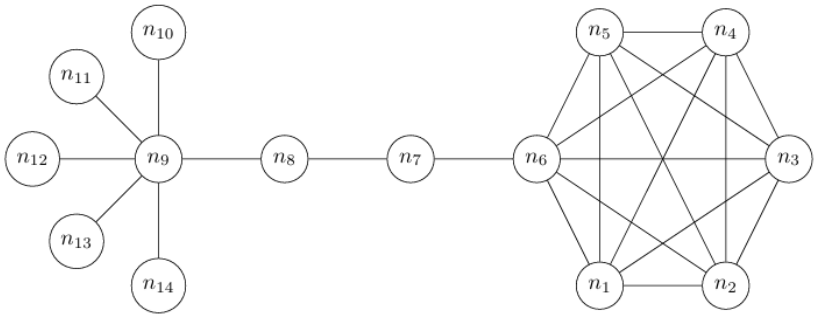


Figure 5

3.1 punto a

Implementando un semplice codice su Python possiamo valutare le centralità dei singoli nodi.

Centralities			
Nodes	Deg	Eig	Inv. dist.
1	0.1087	0.15830326	0.1087
2	0.1087	0.15830326	0.1087
3	0.1087	0.15830326	0.1087
4	0.1087	0.15830326	0.1087
5	0.1087	0.15830326	0.1087
6	0.13043	0.16393047	0.13043
7	0.04348	0.03396326	0.04348
8	0.04348	0.00709314	0.04348
9	0.13043	0.0017546	0.13043
10	0.02174	0.00034844	0.02174
11	0.02174	0.00034844	0.02174
12	0.02174	0.00034844	0.02174
13	0.02174	0.00034844	0.02174
14	0.02174	0.00034844	0.02174

Dalla precedente tabella si nota che i nodi 1:5 hanno la stesso centralità in tutti i casi, lo stesso vale per i nodi 10:14. Ciò è una conseguenza del fatto che la topologia del grafo è invariante per permutazioni tra questi gruppi di nodi. Inoltre, la degree centrality e la invariant distribution centrality coincidono, infatti il grafo considerato è un grafo semplice.

3.2 punto b

Per valutare la Katz centrality è stata uata la libreria numpy (codice in appendice), le centralità risultanti normalizzate sono le seguenti:

Nodo	Katz
1	0.12326782
2	0.12326782
3	0.12326782
4	0.12326782
5	0.12326782
6	0.12986722
7	0.04569536
8	0.03351863
9	0.04555341
10	0.02580526
11	0.02580526
12	0.02580526
13	0.02580526
14	0.02580526

Table 1: Katz centrality.

3.3 punto c

L'algoritmo distribuito che segue sfrutta la convergenza del processo dinamico lineare definito come:

$$x_i^{t+1} = (1-\beta)\sum_j P_{ji}x_j^t + \nu_i\beta \tag{42}$$

Questo processo converge alla Bonacich centrality:

$$x_i = (1-\beta)\sum_j P_{ji}x_j + \nu_i\beta \tag{43}$$

All'interno dell'algoritmo viene definita la matrice di adiacenza partendo dai vettori che definiscono rispettivamente gli archi entranti e uscenti ad ogni nodo. Successivamente, partendo da una distribuzione uniforme, la trasformazione lineare sopra definita viene iteramente applicata al vettore, fino a quando due elementi successivi della successione così definita sono sufficientemente vicini $||x^t - x^{t+1}||_2 < \epsilon_0$. Il seguente algoritmo ha un costo computazionale $O(N^2T(\epsilon))$[1], a differenza dell'algoritmo di inversione che invece costerebbe $O(N^3)$. Dato che $T(\epsilon) = O(N)$ sembrerebbe che non ci sia differenza effettiva nell'usare l'uno rispetto all'altro, tuttavia, l'algoritmo distribuito si presta maggiormente ad essere eseguito su più computer contemporaneamente, infatti, il calcolo delle diverse componenti al tempo t+1 è svincolato dalle altre componenti al tempo t+1 quindi ciascun elemento può essere calcolato separatamente da un computer diverso. In particolare il costo diminuisce in maniera inversamente proporzionale al numero di computer.[2]

```
def distributed_page_ranking(Inedges, Outedges):
    #Si definiscono i parametri dell'algoritmo e la matrice
    beta = 0.15
    eps = 0.001
    L = len(Inedges)
    A = np.array([[0.0]*L]*L)
    def f(x):
        return np.transpose(A_n) @ ((1-beta)*x) + beta

    #si normalizzano le righe
    for i in s:
```

[1] Questo costo computazionale vale nel caso di grafo completo, altrimenti si avrebbe $O(|N|\frac{|E|}{|N|}T(\epsilon))$.

[2] Il seguente ragionamento considera un grafo in cui $\frac{|E|}{|V|} >> 1$, infatti, nel caso di grafi sparsi in cui $|E| \simeq |V|$ non si ha nessun vantaggio a distribuire il calcolo su più computer, dato che il costo della trasmissione va a pareggiare il vantaggio nella suddivisione del processo.

```
        for j in s[i]:
            A[i-1,j-1] = 1.0
    for i in range(L):
        a_i = sum(A[i,:])
        for j in range(L):
            A_n[i, j] = A[i,j]/a_i

    #Ciclo in cui viene iterata l'applicazione della funzione
    #che determina l'evoluzione temporale del vettore x

    x_t1 = np.ones(L)
    x_t2 = np.zeros(L)
    j = 0
    #l'operazione viene ripetuta fino a quando cerci criteri di
    #convergenza non sono soddisfatti
    while (np.transpose(x_t1-x_t2) @ (x_t1 -x_t2))>eps and j<100:
        x_t2 = np.copy(x_t1)
        x_t1 = f(x_t1)
        j += 1
    return x_t1/sum(x_t1)
```

Applicando il precedente algoritmo al grafo considerato si ottengono i seguenti valori di centralità:

Nodo	Page ranking cent.
1	0.07655528
2	0.07655528
3	0.07655528
4	0.07655528
5	0.07655528
6	0.09725102
7	0.05081004
8	0.06140949
9	0.20811755
10	0.03992710
11	0.03992710
12	0.03992710
13	0.03992710
14	0.03992710

Table 2: Tabella con valori della Bonacich centrality .

3.4 punto d

I risultati ottenuti per la Degree centrality, la eigenvector centrality e la Katz centrality determinano ordinamenti diversi dei nodi. In particolare, è possibile notare che, mentre secondo l'eig. centrality il nodo 6 è molto più centrale del nodo 9, questo non accade nel caso della degree centrality (e della unif. dist. cent.). Questo diverso ordinamento è giustificato dal fatto che la eig. centrality, a differenza della degree cent., non è determinata solo da quanti archi entrano in un determinato nodo, ma anche da che peso hanno questi archi, ovvero, da quanto centrali sono i nodi da cui questi archi partono. In particolare tutti i nodi collegati a n_9 sono poco centrali, mentre i nodi collegati a n_6 sono molto più centrali.
Per quanto riguarda la Katz centrality, questa determina un ordinamento che è simile a quello determinato dall'eig. centrality, con l'unica differenza che le variazioni di centralità tra i diversi nodi sono minori.
Eccezione è fatta dalla Bonacich centrality, questa infatti favorisce fortemente il nodo 9. La ragione di questo risultato è che la Bonacich centrality determina la centralità di un nodo partendo dalla centralità dei nodi vicini e andando a ripartire quest'ultima equamente tra tutti i nodi a cui questi sono collegati. L'effetto dei nodi 1:5 sul 6 è mediamente quello di un'unico nodo (dato che il sottografo è completo), mentre l'effetto dei nodi 10:14, seppur

questi hanno una centralità più bassa rispetto i nodi 1:5, conta come l'effetto di 5 nodi differenti. In altre parole, una pagina che cita tanti altri siti, seppur sia molto visitata, distribuisce l'attenzione degli utenti su tutti i link citati, mentre una pagina web che rimanda solamente a un altro sito concentra l'attenzione degli utenti totalmente sull'unico sito citato. Questo è il motivo per cui, considerato il comportamento di utenti reali la cui attenzione è limitata, questa centralità si presta maggiormente al page ranking.

A Codice Python

Di seguito il codice python usato per il calcolo delle centralità:

```
s = {
    1 : [2, 3, 4, 5, 6],
    2 : [1, 3, 4, 5, 6],
    3 : [2, 1, 4, 5, 6],
    4 : [2, 3, 1, 5, 6],
    5 : [2, 3, 4, 1, 6],
    6 : [2, 3, 4, 5, 1, 7],
    7 : [6, 8],
    8 : [7, 9],
    9 : [8, 10, 11, 12, 13, 14],
    10 : [9],
    11 : [9],
    12 : [9],
    13 : [9],
    14 : [9]

}

#degree centrality
C_deg = []
for el in s:
    C_deg.append(len(s[el]))
C_deg = np.array(C_deg)/sum(C_deg)

#Array definition
A = np.array([[0.0]*14]*14)

for i in s:
    for j in s[i]:
        A[i-1,j-1] = 1.0
A

#Normalization
A_n = np.copy(A)
for i in range(14):
    a_i = sum(A[i,:])
    for j in range(14):
        A_n[i, j] = A[i,j]/a_i

#eigenvector centrality
eigenvalues, eigenvectors = np.linalg.eig(np.transpose(A))
C_eig = eigenvectors[:,0]/sum(eigenvectors[:,0])
#invar. distr. centrality
eigenvalues_n, eigenvectors_n = np.linalg.eig(np.transpose(A_n))
C_inv= eigenvectors_n[:,0]/sum(eigenvectors_n[:,0])

#Katz centrality
l = eigenvalues[0]
I = np.eye(14)
beta = 0.15
One = np.ones(14)
A_inv = np.linalg.inv(I-((1-beta)/l)*np.transpose(A))
```

```
mu = A_inv @ One
```